

# **“DineUp – Food and Beverage Mobile App for Diner Self Service”**

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

## **BACHELOR OF TECHNOLOGY**

**In**

**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

**&**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**

**By**

**AJAI MANIKANTA                      2520030320**

**AKSHITH REVURU                      2520030041**

**TEJA                                      2520030569**

**Team ID: 21              Section: 7**

Under the Esteemed Guidance of

**Dr. Spandana**

**Department of Computer Science and Engineering**



## **Koneru Lakshmaiah Education Foundation**

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, [www.klh.edu.in](http://www.klh.edu.in)

**K L (Deemed to be) University  
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

**&**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**



### **Declaration**

The Project Report entitled “**Food and Beverage Mobile App for Diner Self Service (DineUp)**” is a record of Bonafide work of **AJAI MANIKANTA - 2520030320, AKSHITH - 2520030041, TEJA - 2520030569** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

**AJAI MANIKANTA – 2520030320**

**AKSHITH – 2520030041**

**TEJA – 2520030569**

**K L (Deemed to be) University**

**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

**&**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**



## **CERTIFICATE**

This is to certify that the project based report entitled “**Food and Beverage Mobile App for Diner Self Service (DineUp)**” is a bonafide work done and submitted by **AJAI MANIKANTA - 2520030320, AKSHITH - 2520030041, TEJA - 2520030569** under the guidance of **Dr. Spandana**, in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Engineering, K L (Deemed to be University), in the course Database Systems Engineering and Distributed Backend Development (25CS1302E), during the academic year **2026-2027**.

**Signature of the Guide**

**Signature of the Course Coordinator**

**Signature of the HOD**

## **ACKNOWLEDGEMENT**

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in the completion of our project.

Our greatest appreciation goes to our Course Coordinator and our guide **Dr. Spandana**, Department of Computer Science and Engineering, whose tremendous support, encouragement and guidance for this project cannot be expressed in words.

We express our gratitude to the ***Head of the Department of Computer Science and Engineering***, KLH Bachupally Campus, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

**AJAI MANIKANTA – 2520030320**

**AKSHITH – 2520030041**

**TEJA – 2520030569**

## **Abstract**

DineUp is a food and beverage self-service ordering platform designed to simplify and digitize the restaurant ordering process by connecting diners, restaurants and administrators through a centralized system. The main objective of DineUp is to reduce dependency on traditional manual ordering methods and provide customers with a convenient digital platform through which they can browse available restaurants, select a restaurant branch, enter or select their table number, view the available digital menu, check dish prices and availability, add items to a cart, place orders and track the status of their orders. The system also provides a dedicated Restaurant Web Dashboard through which restaurant users can manage their restaurant information, branches, menu items, dish images, prices, availability and customer orders, while updating the status of orders during the preparation process. A separate Admin Web Dashboard provides centralized management of customers, restaurants and orders, allowing the administrator to monitor and manage important platform information.

DineUp follows a centralized client-server architecture in which the Customer application, Restaurant Web and Admin Web communicate with a Node.js and Express.js REST API backend. The backend performs authentication, request validation, business logic, restaurant and menu management, order processing and database operations. TiDB Cloud is used as the centralized SQL database for storing customer, restaurant, branch, menu, order and order-item information. Firebase Hosting is used for hosting the web interfaces, while the backend is deployed using Render.

The system also supports restaurant, branch and dish image management so that uploaded images can be displayed to customers along with the corresponding restaurant or menu information. The database design uses relational entities such as Customers, Restaurants, Branches, Menu Items, Orders and Order Items with primary and foreign key relationships to maintain data consistency. The project demonstrates how database systems, distributed backend services and web/mobile interfaces can be integrated to provide a practical digital restaurant self-service solution. The current project is developed as an academic prototype and uses cash payment as the demonstration payment method, while additional payment and notification features can be considered as future enhancements.

## Index

| S. No. | Chapters                 | Topics                                                                                                                                                                                                                          | Page.no |
|--------|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
|        |                          | Acknowledgement                                                                                                                                                                                                                 |         |
|        |                          | Abstract                                                                                                                                                                                                                        |         |
| 1      | Introduction             | 1.1 Background of the project<br>1.2 Problem statement<br>1.3 Scope of the project<br>1.4 Objective(s)<br>1.5 Importance of the application<br>1.6 Target users/audience                                                        | 5-9     |
| 2      | System Requirements      | 2.1 Hardware requirements<br>2.2 Software requirements<br>2.3 Development tools and frameworks                                                                                                                                  | 10-11   |
| 3      | Technology Stack         | 3.1 Front-end (HTML, CSS, JavaScript)<br>3.2 Back-end (Node.js, Express.js)<br>3.3 Database (TiDB Cloud, MySQL-compatible SQL)<br>3.4 Version Control (Git, GitHub)<br>3.5 APIs or External services (Firebase Hosting, Render) | 12-13   |
| 4      | System Architecture      | 4.1 High-level architecture diagram<br>4.2 Description of each layer (frontend, backend, database)<br>4.3 Deployment architecture (Firebase, Render, TiDB Cloud)                                                                | 14-15   |
| 5      | Design                   | 5.1 ER Diagram / Database schema                                                                                                                                                                                                | 16-19   |
| 6      | Implementation           | 6.1 Module-wise implementation<br>6.2 Front-end logic (UI rendering, state management)<br>6.3 API integration<br>6.4 Authentication & Authorization                                                                             | 20-22   |
| 7      | Features                 | 7.1 List of main features<br>7.2 How each feature works (user perspective + technical description)                                                                                                                              | 23-24   |
| 8      | Testing                  | 8.1 Postman testing (frontend/backend)<br>8.2 Integration testing<br>8.3 Tools used for testing<br>8.4 Test cases and results                                                                                                   | 25-26   |
| 9      | Deployment               | 9.1 Steps to deploy<br>9.2 Environment configuration<br>9.3 Hosting of frontend and backend                                                                                                                                     | 27      |
| 10     | Challenges & Limitations | 10.1 Issues faced during development<br>10.2 Solutions applied<br>10.3 Current limitations or known bugs                                                                                                                        | 28      |

|    |                     |                                                                                                                                                                                          |       |
|----|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 11 | Future Enhancements | 11.1 Planned features<br>11.2 Possible integrations or optimizations                                                                                                                     | 29    |
| 12 | Conclusion          | 12.1 Summary of the project<br>12.2 What was achieved<br>12.3 Skills learned during development                                                                                          | 30-31 |
| 13 | References          | - Books, tutorials, APIs, documentation sites used                                                                                                                                       | 32    |
| 14 | Appendices          | - Screenshots of the app<br>- Sample code snippets<br>- Installation/setup instructions<br>- User manual or guide<br>- Geo Tag photos with guide<br>- Review forms with guide signatures | 33-38 |

## CHAPTER -1 INTRODUCTION

### 1.1 Background of the Project

The food and beverage service industry has changed considerably with the spread of smartphones, web technologies and cloud computing. Customers today are accustomed to ordering products, booking rides and tracking deliveries through digital applications. In contrast, the process of ordering food inside a restaurant has remained largely traditional. A diner usually waits to be seated, waits again for a physical or static menu, depends on a waiter to note down the order, and then has very little visibility of how far the preparation has progressed.

This traditional process works reasonably well when a restaurant is quiet, but it becomes inefficient during busy periods. Waiters have to handle many tables at the same time, orders may be communicated incorrectly or recorded late, menu changes such as unavailable dishes or revised prices are not always known to customers, and the restaurant has no unified digital record of its orders. For restaurants that operate several branches the problem is multiplied, because every branch has to be managed separately.

The idea of DineUp emerged from these observations. DineUp is a food and beverage self-service ordering platform that connects diners, restaurants and administrators through one centralized system. Using the DineUp customer interface, a diner can browse restaurants, select a branch, enter or select the table number, view the digital menu with dish images, prices and availability, add dishes to a cart, place the order and track its status. Restaurants receive these orders on a dedicated web dashboard, and an administrator manages platform-level information through a separate admin dashboard.

From a technical perspective, the project integrates frontend interfaces, a REST API backend, a relational cloud database and cloud hosting. The Customer application, the Restaurant Web and the Admin Web communicate with a Node.js and Express.js backend, which performs authentication, request validation, business logic and database operations. The data is stored in TiDB Cloud, a MySQL-compatible SQL database, the web interfaces are hosted on Firebase Hosting and the backend is deployed on Render. Because of this, the deployed system does not depend on a development laptop being switched on.

The project is carried out as part of the course Database Systems Engineering and Distributed Backend Development (25CS1302E). From an academic and engineering perspective, it combines several core areas of computer science:

1. Database Management: relational design with primary and foreign keys for customers, restaurants, branches, menu items, orders and order items.
2. Distributed Backend Development: a centralized REST API consumed by several independent clients and deployed on the cloud.
3. Web Technology: HTML5, CSS3 and JavaScript interfaces for customers, restaurants and administrators.

4. Security: JWT-based authentication, password hashing and server-side validation of every request.
5. Software Engineering Principles: modular design, role separation, API testing with Postman and version control with Git and GitHub.

To summarize, the background of this project rests on three fundamental observations:

1. Traditional restaurant ordering is dependent on waiters and offers customers limited visibility of their order.
2. Modern web and cloud technologies make a centralized self-service ordering platform practical to build and deploy.
3. Customers, restaurants and administrators have different needs and therefore need dedicated interfaces working on one shared, consistent database.

## **1.2 Problem Statement**

Traditional restaurant ordering systems can involve the following difficulties:

- Manual menu viewing
- Waiter-dependent ordering
- Manual recording of customer orders
- Difficulty managing large numbers of orders
- Limited visibility of order status
- Separate or fragmented restaurant information
- Difficulty maintaining menu availability and pricing
- Manual management of restaurant and customer records

Because of these issues, customers experience delays and uncertainty, while restaurants find it hard to manage a large number of orders and to keep menu information up to date. Restaurant details, branch details, menu data and customer records are often maintained separately, which makes them difficult to manage as the business grows. Therefore, there is a need for a centralized digital self-service system that allows customers to place orders directly, while providing restaurants and administrators with dedicated management interfaces.

### ***Existing System***

In a traditional restaurant environment:

- Customers may depend on waiters to place orders.
- Menus may be physical or static.
- Orders may be manually recorded.
- Customers may have limited visibility of order progress.
- Restaurant information may be managed separately.

- Menu availability and pricing may require manual updates.
- Customer and order records can become difficult to manage as the restaurant grows.

### ***Proposed System***

DineUp replaces this manual process with a centralized digital system that provides:

- Digital restaurant browsing, branch selection and table-based ordering
- Digital menus with dish images, prices and availability information
- Shopping cart, digital order placement and order status tracking
- Restaurant management dashboard and admin management dashboard
- Centralized REST API and centralized SQL database
- Cloud-based deployment
- Restaurant, branch and dish image management

### **1.3 Scope of the Project**

The scope of the DineUp project covers the design and development of a working academic prototype with the following components:

- A Customer module through which diners browse restaurants, select a branch and table, view menus, build a cart, place orders and track order status.
- A Restaurant Web Dashboard through which restaurant users manage restaurant information, branches, menu items, prices, availability, images and customer orders.
- An Admin Web Dashboard through which the administrator manages customers, restaurants and orders.
- A Node.js and Express.js REST API backend that handles authentication, validation, business logic, image handling and database operations.
- A relational database on TiDB Cloud that stores customers, restaurants, branches, menu items, orders and order items.
- Cloud deployment using Firebase Hosting for the web interfaces, Render for the backend and TiDB Cloud for the database.
- Cash as the demonstration payment method.

The following items are outside the scope of the current prototype and are listed as future enhancements: real online payment transactions, advanced analytics, production-grade notification infrastructure and packaging of the customer web experience into a dedicated Android APK/AAB.

## 1.4 Objective(s)

**Objective 1 - Customer Self-Service.** To provide customers with a simple digital platform for browsing restaurants, branches, tables and menus and placing food orders without depending completely on manual ordering.

**Objective 2 - Restaurant Management.** To provide restaurants with a dedicated dashboard for managing restaurant information, branches, dishes, prices, availability, images and customer orders.

**Objective 3 - Centralized Administration.** To provide an administrator with a centralized interface for managing customers, restaurants and orders.

**Objective 4 - Database Management.** To design and implement a relational database that maintains customer, restaurant, branch, menu and order information using primary and foreign key relationships.

**Objective 5 - Backend Integration.** To develop a centralized REST API backend that connects all three interfaces and handles authentication, validation, business logic and database operations.

**Objective 6 - Cloud Deployment.** To deploy the frontend and backend components using cloud-based hosting so that the system can operate independently of the development laptop.

## 1.5 Importance of the Application

**For diners:** DineUp improves convenience. A customer can see the complete menu with images, prices and availability before ordering, can place the order directly from the table, and can follow the order status instead of waiting without information. This reduces dependence on manual ordering and reduces communication errors.

**For restaurants:** The Restaurant Web Dashboard gives one place to manage the restaurant profile, branches, dishes, prices, availability and incoming orders. Updating a dish price or switching a dish to unavailable is immediately reflected for customers, which keeps menu information accurate.

**For administrators:** The Admin Web Dashboard provides centralized visibility of customers, restaurants and orders, so that platform-level information can be monitored and managed from a single interface.

**Academically:** The project demonstrates how database systems, distributed backend services and web or mobile interfaces can be integrated into a practical solution to a real-world problem.

### *Feasibility Study*

**Technical Feasibility.** The system uses widely available technologies such as HTML, CSS, JavaScript, Node.js, Express.js and SQL. These technologies support REST-based client-server communication and cloud deployment.

**Economic Feasibility.** The project uses open-source development technologies and available cloud and free-tier resources, making it suitable for an academic prototype without requiring expensive infrastructure.

**Operational Feasibility.** The system separates customer, restaurant and administrator functions into dedicated interfaces, which makes the workflow easier to understand and manage.

**Schedule Feasibility.** The project is divided into independent modules, namely frontend development, backend development, database design, authentication, order management and deployment, which allows the components to be developed and tested progressively.

## 1.6 Target Users/Audience

The primary users of DineUp are:

- **Diners / Customers** who want to browse restaurants, view menus and place and track orders digitally.
- **Restaurant owners, managers and staff** who manage restaurant information, branches, menus and incoming orders through the Restaurant Web Dashboard.
- **Platform administrators** who manage customers, restaurants and orders through the Admin Web Dashboard.

The secondary audience includes faculty members and evaluators who review the project as an academic prototype, and future developers who may extend DineUp with additional payment options, notifications, analytics and a dedicated Android application.

## CHAPTER -2 SYSTEM REQUIREMENTS

### 2.1 Hardware Requirements

Hardware requirements define the physical resources needed to build, test and run the application. Because DineUp is deployed on cloud platforms (Firebase Hosting, Render and TiDB Cloud), no dedicated server hardware is required by the team. The requirements therefore fall into two categories: those needed by the developers, and those needed by the end users of the three interfaces.

#### *Development machine (recommended)*

| Component    | Recommended Specification                                                                      |
|--------------|------------------------------------------------------------------------------------------------|
| Processor    | Intel Core i3 / AMD Ryzen 3 or higher                                                          |
| Memory (RAM) | 8 GB (minimum 4 GB)                                                                            |
| Storage      | 256 GB SSD with at least 10 GB free space                                                      |
| Display      | 1366 x 768 or higher                                                                           |
| Network      | Stable broadband internet connection (required for cloud database, hosting and Git operations) |
| Peripherals  | Keyboard, mouse; Android smartphone for testing the customer interface                         |

#### *End-user devices*

- **Customer:** an Android smartphone (or any smartphone or computer with a modern browser) with internet access, since the Customer interface is planned as an APK/AAB and is also available as a web application.
- **Restaurant:** a laptop, desktop or tablet with a modern web browser to use the Restaurant Web Dashboard at the counter or kitchen.
- **Administrator:** a laptop or desktop with a modern web browser to use the Admin Web Dashboard.

### 2.2 Software Requirements

| Category          | Requirement                                                        |
|-------------------|--------------------------------------------------------------------|
| Operating System  | Windows 10/11, Linux or macOS for development                      |
| Runtime           | Node.js (LTS version) with npm package manager                     |
| Backend Framework | Express.js (REST API)                                              |
| Database          | TiDB Cloud (MySQL-compatible SQL)                                  |
| Web Browser       | Google Chrome, Microsoft Edge or Mozilla Firefox (latest versions) |
| Hosting Accounts  | Firebase (Hosting), Render (backend), TiDB Cloud (database)        |
| Version Control   | Git and a GitHub account                                           |
| API Testing       | Postman                                                            |
| Diagram Tool      | diagrams.net (draw.io)                                             |
| Mobile Packaging  | Android toolchain for APK/AAB packaging (planned)                  |

## 2.3 Development Tools and Frameworks

**Visual Studio Code** is used as the primary code editor for writing the HTML, CSS, JavaScript and Node.js source code. Its integrated terminal, Git integration and extensions make it suitable for full-stack development.

**Node.js and Express.js** form the backend runtime and framework. Express.js simplifies routing, middleware handling, request parsing and error handling for the REST API.

**Git and GitHub** are used for version control and for sharing the source code among the three team members. GitHub also connects with Render so that backend updates can be deployed from the repository.

**Postman** is used to send requests to the REST APIs and to verify responses for authentication, restaurant management, menu management and order processing.

**diagrams.net** is used to draw the system architecture diagram and the entity relationship diagram that are included in this report.

**Firebase CLI** is used to publish the Customer Web, Restaurant Web and Admin Web interfaces to Firebase Hosting.

## CHAPTER -3 TECHNOLOGY STACK

### 3.1 Front-end

The front-end of DineUp is built using HTML5, CSS3 and JavaScript. Three separate interfaces are provided: the Customer interface (APK / Web), the Restaurant Web Dashboard and the Admin Web Dashboard. All three communicate with the same backend through REST API calls.

**HTML5** defines the structure of the pages: restaurant lists, branch and table selection, menus, cart, order status screens and the management forms used by restaurants and administrators.

**CSS3** provides layout, colours, typography and responsive design so that the interfaces remain usable on smartphones, tablets and desktop screens. Clear visual presentation is especially important for the customer interface, because dish images and prices must be easy to read.

**JavaScript** adds interactivity. It sends requests to the backend, receives JSON responses, updates the page with restaurants, menus and orders, maintains the cart (adding dishes, changing quantities and calculating the total) and handles form submissions such as login, menu updates and image uploads.

### 3.2 Back-end

The backend is the central component of DineUp and is developed using Node.js with the Express.js framework. Node.js executes JavaScript on the server using an event-driven, non-blocking model, which is well suited to an API that serves many short requests from several clients. Express.js adds routing and middleware on top of Node.js.

The backend provides REST APIs for authentication, customer management, restaurant management, branch management, menu management, order creation, order retrieval, order status updates, database operations, image upload handling and data validation. It acts as the communication layer between the frontend applications and the database, so that no client connects to the database directly.

### 3.3 Database

DineUp uses TiDB Cloud, which provides a MySQL-compatible SQL database environment. A relational database is a natural choice for DineUp because the data is highly structured and interrelated: customers place orders, orders contain items, items refer to menu items, and menu items belong to restaurants.

The database stores information related to customers, users, restaurants, branches, menu items, orders, order items, order status history, restaurant requests, support information and administrator information. For the main ER diagram, the important entities are CUSTOMERS, RESTAURANTS, BRANCHES, MENU\_ITEMS, ORDERS and ORDER\_ITEMS, connected by primary and foreign key relationships that keep the data consistent.

### 3.4 Version Control

Git is used as the distributed version control system and GitHub as the cloud repository. Version control allows the three team members to work on different modules in parallel, track every change, review history and recover earlier versions when required. The GitHub repository is also the source from which the backend is deployed to Render.

### 3.5 APIs or External Services

DineUp relies on the following cloud services:

- **Firebase Hosting** hosts the Customer Web, Restaurant Web and Admin Web interfaces.
- **Render** hosts the Node.js and Express.js backend as a permanently available service.
- **TiDB Cloud** hosts the centralized SQL database.
- **Backend Image Storage** stores restaurant images, branch images and dish images uploaded through the backend.

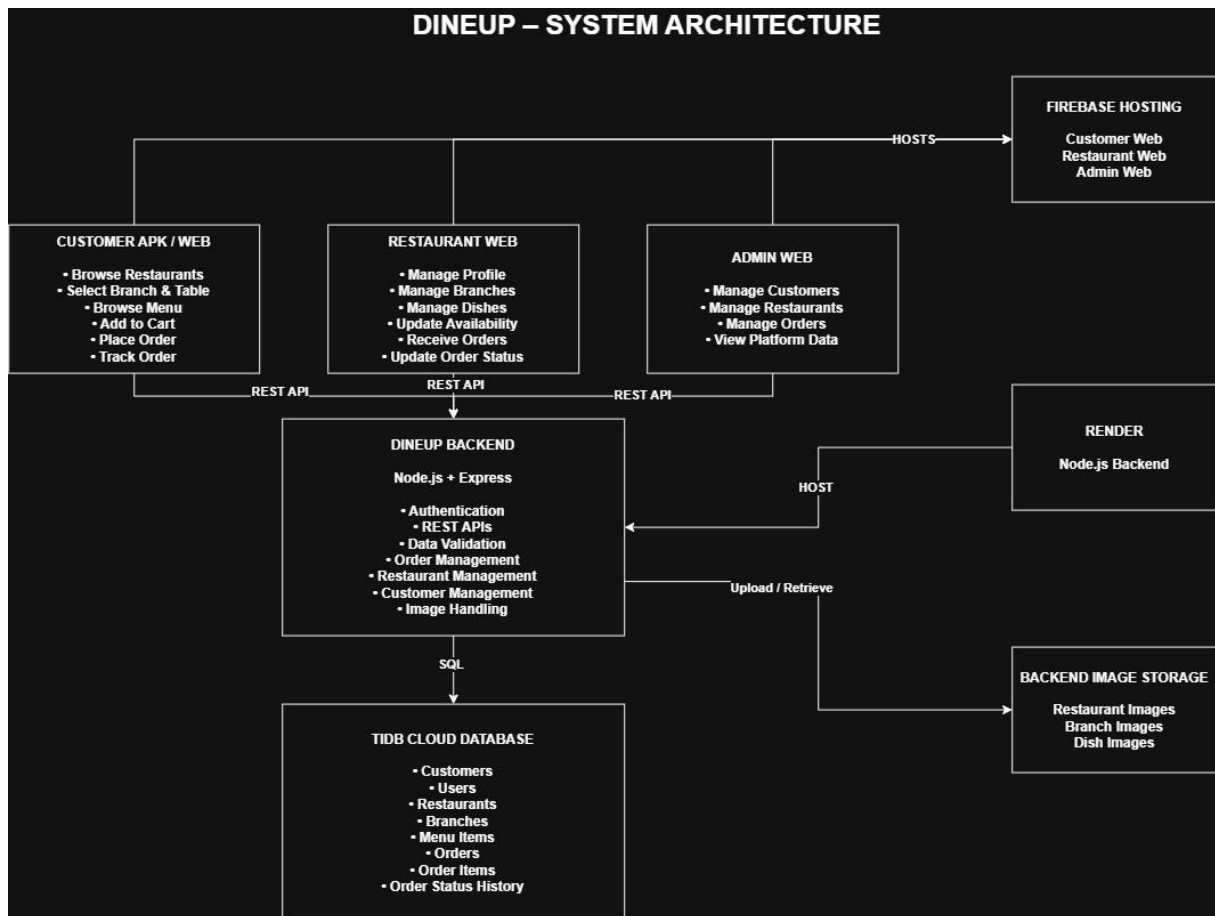
For the current academic demonstration, the payment method is cash, so no external payment gateway is part of the technology stack.

#### *Technology Stack Summary*

| Layer          | Technology                                  | Purpose                                        |
|----------------|---------------------------------------------|------------------------------------------------|
| Frontend       | HTML5, CSS3, JavaScript                     | Customer, Restaurant and Admin interfaces      |
| Backend        | Node.js, Express.js, REST API               | Business logic, validation, image handling     |
| Database       | TiDB Cloud (MySQL-compatible SQL)           | Centralized relational data storage            |
| Authentication | JWT, password hashing                       | Secure login and protected routes              |
| Hosting        | Firebase Hosting, Render                    | Cloud deployment of web interfaces and backend |
| Tools          | VS Code, Git, GitHub, Postman, diagrams.net | Development, versioning, testing, diagrams     |
| Mobile         | Android APK/AAB (planned)                   | Packaging of the customer application          |

## CHAPTER -4 SYSTEM ARCHITECTURE

### 4.1 High-level architecture diagram



### 4.2 Description of Each Layer (Frontend, Backend, Database)

The architecture of DineUp follows a centralized client-server model. Three independent clients communicate with a single backend through REST APIs, and the backend alone communicates with the database and the image storage. This separation keeps the user interfaces, business logic and data management independent of one another.

**Client (Frontend) Layer.** This layer consists of three interfaces. The *Customer APK / Web* lets diners browse restaurants, select a branch and table, browse the menu, add dishes to the cart, place an order and track it. The *Restaurant Web* lets restaurant users manage their profile, branches and dishes, update availability, receive orders and update order status. The *Admin Web* lets the administrator manage customers, restaurants and orders and view platform data.

**Hosting Layer for the Web Interfaces.** The Customer Web, Restaurant Web and Admin Web are hosted on Firebase Hosting, shown in the diagram with the HOSTS link to the three interfaces.

**Backend Layer.** The DineUp Backend, built with Node.js and Express, is the heart of the system. It provides authentication, REST APIs, data validation, order management, restaurant management, customer management and image handling. Every request from the three clients passes through this layer, where it is authenticated and validated before any data is read or written. The backend is deployed on Render, which hosts the Node.js backend as shown in the diagram.

**Database Layer.** The TiDB Cloud database stores customers, users, restaurants, branches, menu items, orders, order items and order status history. The backend communicates with the database using SQL queries.

**Image Storage.** Restaurant images, branch images and dish images are uploaded by restaurant users through the backend. The backend stores and retrieves these images (the Upload / Retrieve link in the diagram) so that the Customer interface can display them along with the corresponding restaurant or menu information.

#### 4.3 Deployment architecture

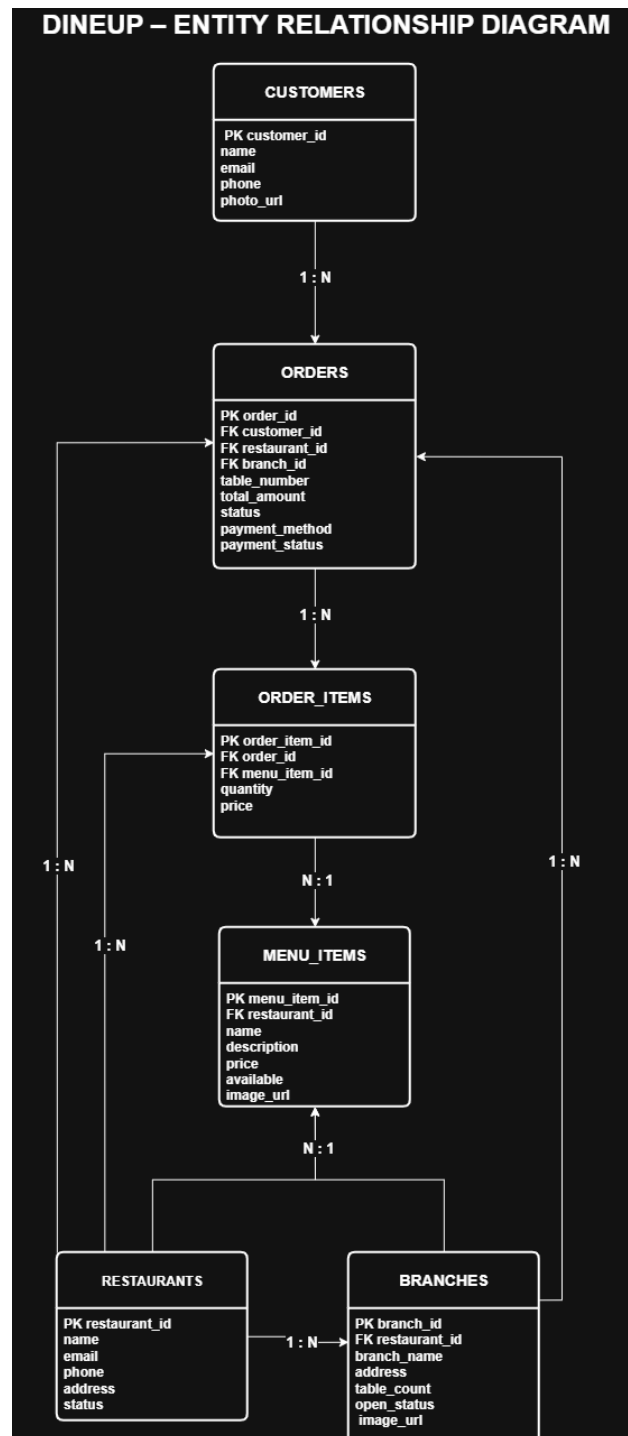
DineUp is designed so that the development laptop does not need to remain switched on for the application to be accessed. The deployment architecture consists of three cloud components:

| Component                               | Deployed On           | Role                                       |
|-----------------------------------------|-----------------------|--------------------------------------------|
| Customer Web, Restaurant Web, Admin Web | Firebase Hosting      | Serves the three user interfaces           |
| Node.js + Express.js Backend            | Render                | Permanent REST API layer                   |
| SQL Database                            | TiDB Cloud            | Centralized relational storage             |
| Restaurant, Branch and Dish Images      | Backend Image Storage | Uploaded and retrieved through the backend |

In this architecture the frontend communicates with the permanent backend API, and the backend communicates with the cloud database. The customer application is currently available as a web interface and is planned to be packaged as an APK/AAB as a future step.

## CHAPTER -5 DESIGN

### 5.1 ER Diagram / Database schema



The Entity Relationship diagram above shows the main entities of DineUp and the relationships between them. The database design uses relational entities with primary keys (PK) and foreign keys (FK) to maintain data consistency. The main entities are described below.

### ***CUSTOMERS***

Stores customer profile information.

| Attribute   | Key | Description                            |
|-------------|-----|----------------------------------------|
| customer_id | PK  | Unique identifier of the customer      |
| name        | -   | Name of the customer                   |
| email       | -   | Email address of the customer          |
| phone       | -   | Contact number of the customer         |
| photo_url   | -   | Location of the customer profile photo |

### ***RESTAURANTS***

Stores restaurant information.

| Attribute     | Key | Description                         |
|---------------|-----|-------------------------------------|
| restaurant_id | PK  | Unique identifier of the restaurant |
| name          | -   | Name of the restaurant              |
| email         | -   | Email address of the restaurant     |
| phone         | -   | Contact number of the restaurant    |
| address       | -   | Address of the restaurant           |
| status        | -   | Current status of the restaurant    |

### ***BRANCHES***

Stores the individual branches of a restaurant.

| Attribute     | Key | Description                              |
|---------------|-----|------------------------------------------|
| branch_id     | PK  | Unique identifier of the branch          |
| restaurant_id | FK  | Restaurant to which the branch belongs   |
| branch_name   | -   | Name of the branch                       |
| address       | -   | Address of the branch                    |
| table_count   | -   | Number of tables available in the branch |
| open_status   | -   | Whether the branch is open               |
| image_url     | -   | Location of the branch image             |

### ***MENU\_ITEMS***

Stores the dishes/menu items belonging to restaurants.

| Attribute     | Key | Description                             |
|---------------|-----|-----------------------------------------|
| menu_item_id  | PK  | Unique identifier of the menu item      |
| restaurant_id | FK  | Restaurant to which the dish belongs    |
| name          | -   | Name of the dish                        |
| description   | -   | Description of the dish                 |
| price         | -   | Price of the dish                       |
| available     | -   | Whether the dish is currently available |
| image_url     | -   | Location of the dish image              |

## ***ORDERS***

Stores customer order information.

| Attribute      | Key | Description                                                 |
|----------------|-----|-------------------------------------------------------------|
| order_id       | PK  | Unique identifier of the order                              |
| customer_id    | FK  | Customer who placed the order                               |
| restaurant_id  | FK  | Restaurant receiving the order                              |
| branch_id      | FK  | Branch receiving the order                                  |
| table_number   | -   | Table from which the order is placed                        |
| total_amount   | -   | Total amount of the order                                   |
| status         | -   | Order status (for example NEW, PREPARING, READY, COMPLETED) |
| payment_method | -   | Payment method (CASH for the demonstration)                 |
| payment_status | -   | Payment status of the order                                 |

## ***ORDER\_ITEMS***

Stores the individual dishes included in each order.

| Attribute     | Key | Description                           |
|---------------|-----|---------------------------------------|
| order_item_id | PK  | Unique identifier of the order item   |
| order_id      | FK  | Order to which the item belongs       |
| menu_item_id  | FK  | Dish that was ordered                 |
| quantity      | -   | Quantity of the dish ordered          |
| price         | -   | Price of the dish for this order item |

### ***ER Relationships***

| <b>Relationship</b>      | <b>Cardinality</b> | <b>Meaning</b>                                   |
|--------------------------|--------------------|--------------------------------------------------|
| CUSTOMERS - ORDERS       | 1 : N              | One customer can place multiple orders           |
| RESTAURANTS - BRANCHES   | 1 : N              | One restaurant can have multiple branches        |
| RESTAURANTS - MENU_ITEMS | 1 : N              | One restaurant can have multiple menu items      |
| RESTAURANTS - ORDERS     | 1 : N              | One restaurant can receive multiple orders       |
| BRANCHES - ORDERS        | 1 : N              | One branch can receive multiple orders           |
| ORDERS - ORDER_ITEMS     | 1 : N              | One order can contain multiple order items       |
| MENU_ITEMS - ORDER_ITEMS | 1 : N              | One menu item can appear in multiple order items |

Apart from these six main entities, the database also stores supporting information such as users, order status history, restaurant requests, support information and administrator information, as shown in the system architecture diagram. The foreign keys ensure that an order always refers to a valid customer, restaurant and branch, and that every order item refers to a valid order and a valid menu item.

## CHAPTER -6 IMPLEMENTATION

### 6.1 Module-wise Implementation

DineUp is implemented as four cooperating modules: the Customer module, the Restaurant module, the Admin module and the Backend module. The database and image management are shared by all of them. Each module was designed to work independently while using the same REST API and the same database.

#### A. Customer Module

The Customer module is the main diner-facing component. Customers can sign in, manage their profile, browse restaurants, view restaurant information, select a restaurant branch, select or enter table information, view available dishes with images, prices and availability, add dishes to the cart, modify quantities, view the cart total, place an order, select the demonstration payment method and track the order status. The complete customer workflow is shown below.

**Customer Login → Customer Profile → Browse Restaurants → Select Restaurant → Select Branch → Select Table → View Menu → Select Dishes → Add to Cart → Review Cart → Place Order → Restaurant Receives Order → Restaurant Updates Status → Customer Tracks Order**

#### B. Restaurant Module

The Restaurant Web Dashboard is used by restaurant users. Restaurants log in using credentials provided through the platform and can manage restaurant information and the restaurant profile image, manage branches and upload branch images, manage menu items (names, descriptions, prices, dish images), enable or disable dish availability, manage branch status, view customer orders and update order status.

**Restaurant Login → Restaurant Dashboard → Manage Restaurant → Manage Branches → Manage Menu Items → Upload Dish / Branch Images → Set Prices and Availability → Receive Customer Orders → Update Order Status**

#### C. Admin Module

The Admin Web Dashboard provides centralized platform management. The administrator can manage customers and restaurants, view restaurant information, manage restaurant requests and information, view orders, manage platform-level data, delete customer records or orders where required, and monitor the information stored in the system.

**Admin Login → Admin Dashboard → Manage Customers → Manage Restaurants → View Orders → Manage Platform Information**

#### ***D. Backend Module***

The backend, developed with Node.js and Express.js, provides REST APIs for authentication, customer management, restaurant management, branch management, menu management, order creation, order retrieval, order status updates, database operations, image upload handling and data validation. It is the only component that talks to the database.

#### ***E. Image Management***

Restaurant users can upload restaurant images, branch images and dish/menu images. The images are handled by the backend, stored in the backend image storage and associated with their respective database records through the `image_url` attribute of the corresponding table. The customer interface then displays the correct restaurant, branch and dish images while browsing, which makes the digital menu more visually informative than a simple text-based menu.

### **6.2 Front-end Logic (UI rendering, state management)**

Each interface is a set of HTML pages styled with CSS and driven by JavaScript. When a page is opened, JavaScript requests the required data from the backend (for example the list of restaurants, the branches of a restaurant or the menu of a branch), receives it as JSON and renders it into the page. Forms are validated in the browser for a better user experience, but the backend performs its own validation and never trusts the values sent by the frontend directly.

In the Customer interface, the selected restaurant, branch and table number identify the context of the order. The cart keeps the selected dishes and their quantities and recalculates the total whenever a dish is added, removed or its quantity is changed. When the customer confirms the order, the cart contents are sent to the backend, which validates them and stores the order. The order status screen then fetches the latest status of the order from the backend.

In the Restaurant and Admin interfaces, tables and forms are rendered from the data returned by the API. Actions such as updating a price, changing availability, uploading an image, updating an order status or deleting a record are sent to the corresponding API and the page is refreshed with the result.

### **6.3 API Integration**

All three interfaces integrate with the backend through REST APIs. Requests and responses use JSON, standard HTTP methods are used for reading, creating, updating and deleting resources, and image uploads are sent to the backend upload handler. After login, the authentication token is attached to requests so that the backend can identify the user and role. The main API groups are summarized below.

| API Group           | Responsibility                                                                         |
|---------------------|----------------------------------------------------------------------------------------|
| Authentication      | Login for customers, restaurants and administrators; token generation and verification |
| Customer Management | Customer profile creation, retrieval, update and deletion by administrator             |

| API Group             | Responsibility                                                                         |
|-----------------------|----------------------------------------------------------------------------------------|
| Restaurant Management | Restaurant information, restaurant profile image, restaurant listing                   |
| Branch Management     | Branch creation, update, status, table count and branch images                         |
| Menu Management       | Menu item creation, update, price, availability and dish images                        |
| Order Management      | Order creation, order retrieval, order status updates, order deletion by administrator |
| Image Handling        | Upload and retrieval of restaurant, branch and dish images                             |
| Validation            | Server-side validation of request data before database operations                      |

## 6.4 Authentication & Authorization

Security in DineUp is enforced by the backend. The system uses JWT-based authentication: after a successful login the backend issues a signed JSON Web Token that identifies the user and the role, and protected routes accept only requests carrying a valid token. Passwords are stored using password hashing and never as plain text.

Authorization follows role-based access concepts. A customer can access customer functions, a restaurant user can manage only restaurant functions, and an administrator can access platform-level management functions. Restaurant users log in with credentials provided through the platform.

The backend also performs server-side input validation and database validation. In particular, order validation is done on the backend side: the backend is responsible for validating requests instead of trusting frontend values directly. Together these measures include JWT-based authentication, password hashing, authenticated API routes, server-side input validation, database validation, role-based access concepts and backend-side order validation.

## CHAPTER -7 FEATURES

### 7.1 List of Main Features

1. **Restaurant and branch browsing:** customers can view restaurants, restaurant information and branches.
2. **Table-based ordering:** customers select or enter the table number for the order.
3. **Digital menu:** dishes are shown with images, descriptions, prices and availability.
4. **Shopping cart:** dishes can be added, quantities modified and the cart total viewed.
5. **Order placement:** orders are placed digitally with the demonstration payment method (cash).
6. **Order status tracking:** customers follow the order as the restaurant updates its status.
7. **Restaurant management dashboard:** restaurant profile, branches, menu items, prices, availability and orders.
8. **Image management:** upload and display of restaurant, branch and dish images.
9. **Admin management dashboard:** management of customers, restaurants and orders.
10. **Authentication and security:** JWT-based login, password hashing and server-side validation.

### 7.2 How Each Feature Works (user perspective + technical description)

#### *Restaurant, Branch and Table Selection*

**User Perspective:** The customer opens the application, browses the available restaurants, selects the desired restaurant and branch, and enters or selects the table number.

**Technical Description:** The interface requests restaurants and their branches from the backend. The data comes from the RESTAURANTS and BRANCHES tables, and the chosen restaurant\_id, branch\_id and table number are carried forward to the order.

#### *Digital Menu with Images, Prices and Availability*

**User Perspective:** The customer sees all dishes of the restaurant with pictures, descriptions, prices and an indication of whether each dish is available.

**Technical Description:** The menu is retrieved from the MENU\_ITEMS table for the selected restaurant. The image\_url attribute is used to display the dish image, and the available attribute reflects the restaurant's latest availability setting.

#### *Cart and Order Placement*

**User Perspective:** The customer adds dishes to the cart, changes quantities, reviews the total and places the order, selecting cash as the demonstration payment method.

**Technical Description:** The cart is sent to the backend, which validates the request and creates one record in ORDERS (customer, restaurant, branch, table number, total amount, status, payment method and payment status) and one record in ORDER\_ITEMS for each dish (menu item, quantity and price).

### ***Order Management and Status Tracking***

**User Perspective:** The restaurant sees the new order on its dashboard and updates its status while preparing it. The customer tracks the order status. Example statuses include NEW, PREPARING, READY and COMPLETED.

**Technical Description:** The order system stores the customer, restaurant, branch, table number, ordered dishes, quantity, price, total amount, order status, payment method and payment status. When the restaurant updates the status, the backend updates the order and the customer interface reads the new status through the order retrieval API.

NEW → PREPARING → READY → COMPLETED

### ***Payment for Demonstration***

**User Perspective:** For the current academic demonstration the payment method is CASH. The project is intentionally demonstrated without requiring a real online payment transaction.

**Technical Description:** The payment\_method and payment\_status attributes of the ORDERS table store the payment details, so that additional payment options can be added later without changing the order structure.

### ***Restaurant Management and Image Management***

**User Perspective:** The restaurant updates its profile, branches, dish names, descriptions, prices and availability, and uploads restaurant, branch and dish images.

**Technical Description:** Each change is sent to the backend, validated and stored in the RESTAURANTS, BRANCHES or MENU\_ITEMS tables. Uploaded images are handled by the backend and linked to the records through the image\_url attribute, so that changes are immediately visible to customers.

### ***Admin Management***

**User Perspective:** The administrator views and manages customers, restaurants and orders, and can delete customer records or orders where required.

**Technical Description:** The admin APIs read and modify the CUSTOMERS, RESTAURANTS and ORDERS data, and are accessible only to authenticated users with the administrator role.

## CHAPTER -8 TESTING

### 8.1 Postman Testing (frontend/backend)

The REST APIs of DineUp are tested using Postman and browser requests. Postman allows each API to be called independently of the user interfaces, with chosen request bodies and headers, so that the backend behaviour can be verified before and after the frontend is connected. Requests were grouped according to the API groups of the backend: authentication, customer, restaurant, branch, menu, order and image upload.

For each API, valid requests are tested to confirm that the expected response and database change occur, and invalid requests are tested to confirm that the backend rejects them. Protected routes are tested with and without a valid token to confirm that authentication works. Because the backend validates data itself, tests also confirm that incomplete or incorrect values sent by a client are not accepted.

From the frontend side, the same operations are repeated through the Customer, Restaurant and Admin interfaces in the browser to confirm that the interface correctly calls the API and displays the response.

### 8.2 Integration Testing

Integration testing verifies that the three interfaces, the backend and the database work together. The main integration scenarios are:

1. A restaurant adds a dish with an image and price; the dish appears in the customer menu with the same image and price.
2. A restaurant disables a dish; the customer menu shows it as unavailable.
3. A customer places an order; the order appears on the Restaurant Web Dashboard with the correct branch, table number, dishes and total.
4. The restaurant updates the order status; the customer sees the updated status.
5. The administrator views the customer, restaurant and order records created during the above steps.
6. Foreign-key relationships hold between customers, restaurants, branches, menu items, orders and order items.

### 8.3 Tools Used for Testing

| Tool                      | Use                                                                                         |
|---------------------------|---------------------------------------------------------------------------------------------|
| Postman                   | Testing REST APIs, request validation and authentication                                    |
| Web browsers              | Testing the Customer, Restaurant and Admin interfaces; sending browser requests to the APIs |
| SQL queries on TiDB Cloud | Verifying stored customer, restaurant, branch, menu, order and order-item records           |
| Git / GitHub              | Tracking changes and re-testing after fixes                                                 |

## 8.4 Test Cases and Results

| ID   | Module         | Test Case                                    | Expected Result                                  | Status |
|------|----------------|----------------------------------------------|--------------------------------------------------|--------|
| TC01 | Authentication | Customer login with valid credentials        | Login succeeds and token is issued               | Pass   |
| TC02 | Authentication | Restaurant login with valid credentials      | Restaurant dashboard is accessible               | Pass   |
| TC03 | Authentication | Admin login with valid credentials           | Admin dashboard is accessible                    | Pass   |
| TC04 | Authentication | Login with invalid credentials               | Request rejected with an error message           | Pass   |
| TC05 | Authentication | Protected API called without a token         | Request is denied                                | Pass   |
| TC06 | Restaurant     | Update restaurant profile                    | Updated details are saved and displayed          | Pass   |
| TC07 | Restaurant     | Create / update a branch                     | Branch saved with table count and status         | Pass   |
| TC08 | Restaurant     | Create / update a menu item                  | Item saved with name, description and price      | Pass   |
| TC09 | Restaurant     | Modify the price of a dish                   | New price shown to customers                     | Pass   |
| TC10 | Restaurant     | Enable / disable dish availability           | Availability reflected in the customer menu      | Pass   |
| TC11 | Restaurant     | Upload restaurant, branch and dish images    | Images stored and displayed                      | Pass   |
| TC12 | Customer       | Browse restaurants and branches              | Restaurants and branches are listed              | Pass   |
| TC13 | Customer       | Select branch and enter table                | Branch and table are attached to the order       | Pass   |
| TC14 | Customer       | View menu                                    | Dishes with images, prices and availability      | Pass   |
| TC15 | Customer       | Add, change and remove cart items            | Cart total recalculated correctly                | Pass   |
| TC16 | Customer       | Place an order with CASH payment             | Order and order items are created                | Pass   |
| TC17 | Customer       | Track order status                           | Latest status is displayed                       | Pass   |
| TC18 | Restaurant     | View orders and update status                | Status moves NEW to COMPLETED                    | Pass   |
| TC19 | Admin          | View / manage customers, restaurants, orders | Records displayed; deletion works where required | Pass   |
| TC20 | Database       | Check foreign-key relationships              | Orders and order items refer to valid records    | Pass   |
| TC21 | Validation     | Send incomplete or invalid order data        | Backend rejects the request                      | Pass   |

## CHAPTER -9 DEPLOYMENT

### 9.1 Steps to Deploy

DineUp is deployed in three parts so that the laptop does not need to remain switched on for the deployed application to be accessed. The deployment steps are:

1. **Database:** create the TiDB Cloud database, create the tables for customers, users, restaurants, branches, menu items, orders, order items and order status history with their primary and foreign keys, and note the connection details.
2. **Backend:** push the backend source code to the GitHub repository, create a web service on Render connected to that repository, set the install and start commands for the Node.js application, add the environment variables and deploy. Render provides the permanent URL of the backend API.
3. **Frontend:** set the backend URL in the Customer Web, Restaurant Web and Admin Web interfaces and publish them to Firebase Hosting using the Firebase CLI.
4. **Verification:** open the hosted interfaces and test login, menu display, order placement, order status update and image upload against the deployed backend and database.

### 9.2 Environment Configuration

Sensitive and environment-specific values are kept outside the source code and supplied as environment variables on the hosting platform. Typical configuration values for DineUp are:

| Variable (typical)                        | Purpose                                                  |
|-------------------------------------------|----------------------------------------------------------|
| Database host, port, user, password, name | Connection to the TiDB Cloud database                    |
| JWT secret                                | Signing and verifying authentication tokens              |
| Server port                               | Port on which the Node.js application listens            |
| Allowed origins                           | Web addresses that are permitted to call the backend API |
| Backend URL (frontend setting)            | Address of the Render backend used by the web interfaces |

Database passwords and the JWT secret must never be committed to GitHub.

### 9.3 Hosting of Frontend and Backend

| Part     | Platform         | Description                                                     |
|----------|------------------|-----------------------------------------------------------------|
| Frontend | Firebase Hosting | Hosts the Customer Web, Restaurant Web and Admin Web interfaces |
| Backend  | Render           | Hosts the Node.js + Express.js backend and REST API             |
| Database | TiDB Cloud       | Hosts the MySQL-compatible SQL database                         |

## CHAPTER -10 CHALLENGES & LIMITATIONS

### 10.1 Issues Faced during Development

While designing and building DineUp, the team had to address the following challenges:

1. **Three interfaces on one backend:** the Customer, Restaurant and Admin interfaces needed different functions but had to share the same data and business rules.
2. **Data consistency:** customers, restaurants, branches, menu items, orders and order items are closely related, and inconsistent records (for example an order referring to a missing dish) had to be avoided.
3. **Trust in client data:** values sent from a browser or mobile client, such as prices and totals in an order, cannot be trusted without checking.
4. **Image management:** restaurant, branch and dish images had to be uploaded, stored and linked to the correct records and displayed to customers.
5. **Cloud deployment:** the application had to work without depending on the development laptop, which required coordinating three hosting services.
6. **Scope of payments:** a real online payment transaction would complicate the prototype, so the project scope had to be defined carefully.

### 10.2 Solutions Applied

1. One REST API serves all three interfaces, with role-based access for each user type.
2. A relational schema with primary and foreign keys enforces consistency in TiDB Cloud.
3. The backend validates every request and checks orders on the server side.
4. Uploaded images are handled by the backend and linked to records through the image\_url attribute.
5. Firebase Hosting, Render and TiDB Cloud were used for the frontend, backend and database respectively.
6. Cash was chosen as the demonstration payment method, and the order table stores payment\_method and payment\_status so that other options can be added later.

### 10.3 Current Limitations or Known Bugs

For an honest evaluation of the project, the following limitations of the current prototype are noted:

- The current project is an academic prototype.
- Online payment is not required for the current demonstration.
- Advanced analytics are not included.
- Production-grade notification infrastructure can be added later.
- The customer web experience is intended to be packaged into a dedicated APK as a future step.

## CHAPTER -11 FUTURE ENHANCEMENTS

### 11.1 Planned Features

1. **Dedicated Android APK/AAB for customers:** package the customer experience as an installable Android application.
2. **Push notifications for order updates:** notify customers when the order status changes.
3. **More advanced real-time order communication:** faster delivery of new orders to restaurants and status updates to customers.
4. **Customer order history and personalized recommendations.**
5. **Restaurant analytics and sales reports.**
6. **Advanced admin analytics.**
7. **Additional payment options** if required in a production deployment.

### 11.2 Possible Integrations or Optimizations

1. **Improved cloud image storage** for restaurant, branch and dish images.
2. **Production-scale monitoring and logging** of the backend and database.
3. **Multi-restaurant and multi-branch expansion** to support a larger number of restaurants on the platform.
4. **Notification infrastructure** suitable for production use.

Because DineUp already separates the interfaces, the backend and the database, these enhancements can be added progressively without redesigning the whole system.

## CHAPTER -12 CONCLUSION

### 12.1 Summary of the Project

DineUp demonstrates a centralized food and beverage self-service ordering platform that integrates customer ordering, restaurant management and administrative management into a single system. The customer interface provides diners with a convenient way to browse restaurants and branches, select tables, view digital menus, check dish availability and prices, add items to a cart and place orders. The Restaurant Web Dashboard enables restaurant users to manage their restaurant information, branches, menu items, images, prices, availability and customer orders, while the Admin Web Dashboard provides centralized management of customers, restaurants and orders.

The system is supported by a Node.js and Express.js REST API backend that handles authentication, validation, business logic and communication with the TiDB Cloud relational database. The database uses structured entities and primary/foreign key relationships to maintain consistency between customers, restaurants, branches, menu items and orders. Firebase Hosting and Render provide cloud deployment for the frontend and backend components.

Overall, the project demonstrates the practical application of database systems, distributed backend development and web/mobile technologies to solve a real-world restaurant ordering problem and provides a foundation that can be extended into a dedicated customer Android application and a more advanced production system.

### 12.2 What Was Achieved

The DineUp prototype demonstrates:

- Successful customer-facing restaurant browsing
- Digital menu presentation and dish image display
- Cart functionality and order creation
- Restaurant order management and branch management
- Menu management, image upload and display
- Order status management
- Centralized database storage
- Admin management functionality
- Cloud-hosted frontend and backend architecture

The advantages provided by the system are:

- Reduced dependence on manual ordering and improved customer convenience
- Digital, image-supported menus and better organization of menu data

- Centralized restaurant management and centralized customer and order management
- Structured database design and scalable backend architecture
- Cloud-based deployment
- Separate interfaces for different user roles

### **12.3 Skills Learned during Development**

- Relational database design: ER modelling, primary and foreign keys and SQL on TiDB Cloud.
- Backend development with Node.js and Express.js, including REST API design.
- Authentication and security using JWT, password hashing and server-side validation.
- Frontend development with HTML5, CSS3 and JavaScript and integration with REST APIs.
- Handling and displaying uploaded images linked to database records.
- Cloud deployment using Firebase Hosting, Render and TiDB Cloud.
- API testing with Postman, and teamwork using Git and GitHub.
- Documentation and diagramming with diagrams.net.

## CHAPTER -13 REFERENCES

### Books

Elmasri, R., and Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson Education.

Silberschatz, A., Korth, H. F., and Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education.

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine.

### Tutorials and Learning Resources

MDN Web Docs. *HTML, CSS and JavaScript Documentation*. <https://developer.mozilla.org>

W3Schools. *HTML, CSS and JavaScript Tutorials*. <https://www.w3schools.com>

### APIs and Tools Used

Node.js Foundation. *Node.js Documentation*. <https://nodejs.org/en/docs>

Express.js. *Express - Node.js Web Application Framework*. <https://expressjs.com>

PingCAP. *TiDB Cloud Documentation*. <https://docs.pingcap.com/tidbcloud>

Google. *Firebase Hosting Documentation*. <https://firebase.google.com/docs/hosting>

Render. *Render Documentation*. <https://render.com/docs>

Postman. *Postman Learning Center*. <https://learning.postman.com>

Git. *Git Documentation*. <https://git-scm.com/doc>

diagrams.net. *diagrams.net (draw.io)*. <https://www.drawio.com>

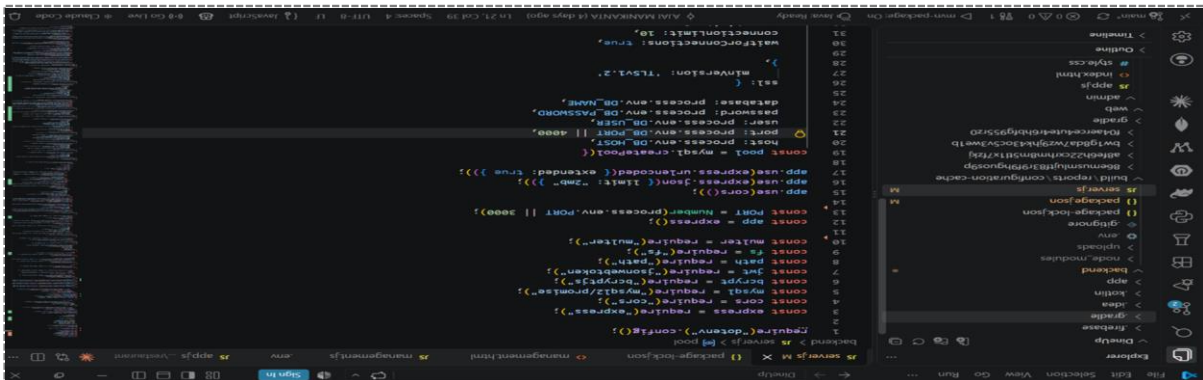
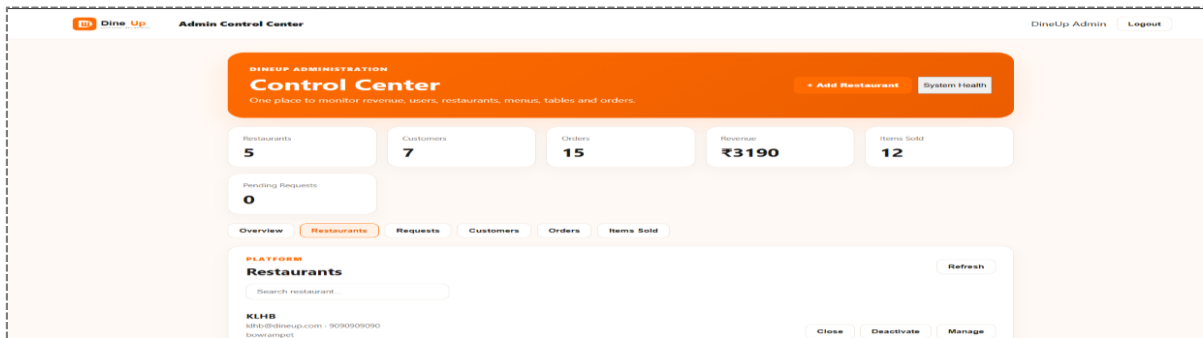
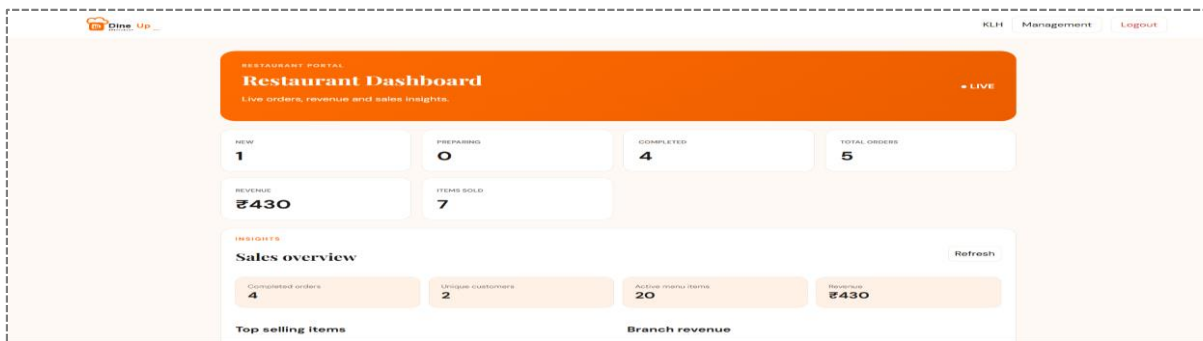
### Documentation and Research References

Jones, M., Bradley, J., and Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519, Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc7519>

## CHAPTER -14 APPENDICES

### Screenshots of the app

The following screenshots of the DineUp customer, restaurant and admin interfaces are to be inserted in the boxes below.



## Sample code snippets

Representative SQL schema of the main DineUp entities, derived from the ER diagram (data types are representative):

```
CREATE TABLE customers (  
  customer_id INT AUTO_INCREMENT PRIMARY KEY,  
  name        VARCHAR(100) NOT NULL,  
  email       VARCHAR(150) NOT NULL,  
  phone       VARCHAR(20),  
  photo_url   VARCHAR(255)  
);  
  
CREATE TABLE restaurants (  
  restaurant_id INT AUTO_INCREMENT PRIMARY KEY,  
  name          VARCHAR(150) NOT NULL,  
  email         VARCHAR(150),  
  phone         VARCHAR(20),  
  address       VARCHAR(255),  
  status        VARCHAR(30)  
);  
  
CREATE TABLE branches (  
  branch_id     INT AUTO_INCREMENT PRIMARY KEY,  
  restaurant_id INT NOT NULL,  
  branch_name   VARCHAR(150) NOT NULL,  
  address       VARCHAR(255),  
  table_count   INT,  
  open_status   BOOLEAN,  
  image_url     VARCHAR(255),  
  FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)  
);  
  
CREATE TABLE menu_items (  
  menu_item_id INT AUTO_INCREMENT PRIMARY KEY,  
  restaurant_id INT NOT NULL,  
  name         VARCHAR(150) NOT NULL,  
  description   TEXT,  
  price        DECIMAL(10,2) NOT NULL,  
  available     BOOLEAN DEFAULT TRUE,  
  image_url    VARCHAR(255),  
  FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)  
);  
  
CREATE TABLE orders (  
  order_id      INT AUTO_INCREMENT PRIMARY KEY,  
  customer_id   INT NOT NULL,  
  restaurant_id INT NOT NULL,  
  branch_id     INT NOT NULL,  
  table_number  INT,  
  total_amount  DECIMAL(10,2),  
  status        VARCHAR(30),          -- NEW / PREPARING / READY / COMPLETED  
  payment_method VARCHAR(30),        -- CASH (demonstration)  
  payment_status VARCHAR(30),  
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id),  
  FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id),  
  FOREIGN KEY (branch_id) REFERENCES branches(branch_id)  
);  
  
CREATE TABLE order_items (  
  order_item_id INT AUTO_INCREMENT PRIMARY KEY,  
  order_id      INT NOT NULL,  
  menu_item_id  INT NOT NULL,  
  quantity      INT NOT NULL,  
  price         DECIMAL(10,2) NOT NULL,  
  FOREIGN KEY (order_id) REFERENCES orders(order_id),  
  FOREIGN KEY (menu_item_id) REFERENCES menu_items(menu_item_id)  
);
```

### Installation/setup instructions

1. Install Node.js (LTS version), Git and Visual Studio Code on the development machine.
2. Clone the project repository from GitHub and open it in Visual Studio Code.
3. Open a terminal in the backend folder and run *npm install* to install the dependencies.
4. Create the TiDB Cloud database, run the SQL scripts to create the tables and note the connection details.
5. Set the environment variables for the database connection and the JWT secret in a local configuration file that is not committed to GitHub.
6. Start the backend server with the project's start command (for example *npm start*) and verify the API with Postman.
7. Open the Customer, Restaurant and Admin web pages in a browser, with the backend URL configured, and test the workflows.
8. To deploy, connect the repository to Render for the backend and publish the web interfaces with the Firebase CLI as described in Chapter 9.

## **User manual or guide**

**Customer:** Log in, browse restaurants, select a restaurant and branch, enter or select your table number, view the menu, add dishes to the cart, adjust quantities, review the total, place the order with the cash payment option and track the order status.

**Restaurant:** Log in with the credentials provided through the platform, update the restaurant profile and branches, add or edit dishes with price, description and image, enable or disable availability, view incoming customer orders and update each order's status as it moves from NEW to PREPARING, READY and COMPLETED.

**Administrator:** Log in to the Admin Web Dashboard, view and manage customers, restaurants and orders, review restaurant information and requests, and delete customer records or orders where required.

### Geo Tag photos with guide

*Insert geo-tagged photo of the team with the guide, Dr. Spandana, here*

**Review forms with guide signatures**

*Insert scanned review forms with guide signatures here*